PES UNIVERSITY

SOFTWARE ENGINEERING LAB

LAB 5

STATIC CODE ANALYSIS

Room Scheduler Application

Name	Khushi Patel
SRN	PES1UG24AM134

1. Introduction

This report documents the static code analysis and code-quality improvement work performed for Lab 5 using the Room Scheduler application. The objective was to inspect the Python source code with Pylint, Bandit and Flake8, identify quality, security and style issues, apply corrections, and verify the updated program.

2. Objectives

- Analyse the Python program using Pylint, Bandit and Flake8.
- Identify code-quality, security and formatting issues reported by the tools.
- Correct the identified issues while preserving the program's functionality.
- Re-run all three tools and record the final results.
- Document the issues, fixes, execution evidence and reflections.

3. Tools and Environment

Tool / Component	Purpose / Version
Python	Python 3.11.8

Pylint	4.1.3 – code quality and static analysis
Bandit	1.9.4 – Python security issue detection
Flake8	7.5.0 – style and formatting checks
Editor / OS	Visual Studio Code on Windows

4. Application Overview

The Room Scheduler application demonstrates room reservations, equipment allocation, reservation cancellation, time-slot validation, schedule export and conflict logging. The main source file submitted for analysis is scheduler_system.py.

5. Initial Static Analysis

The original source was analysed before making corrections. The baseline results identified issues across code quality, security and formatting.

Tool	Initial result	Interpretation
Pylint	5.60/10	22 findings reported
Bandit	2 issues	B605 shell execution (High) and B110 try/except/pass (Low)
Flake8	13 violations	Formatting, blank-line, line-length, bare-except and unused-variable findings

5.1 Baseline Evidence

Figure 1. Original program execution



Figure 2. Pylint before corrections

```
(venv) PS C:\Users\Owner\L5_12_room_scheduler> pylint scheduler_system.py > pylint_before.txt
(venv) PS C:\Users\Owner\L5_12_room_scheduler> Get-Content .\pylint_before.txt
************* Module scheduler_system
scheduler_system.py:18:0: C0301: Line too long (102/100) (line-too-long)
scheduler_system.py:1:0: C0114: Missing module docstring (missing-module-docstring)
scheduler_system.py:9:0: C0103: Constant name "next_reservation" doesn't conform to UPPER_CASE naming style (invalid-name)
scheduler_system.py:11:0: C0116: Missing function or method docstring (missing-function-docstring)
scheduler_system.py:11:0: W0102: Dangerous default value [] as argument (dangerous-default-value)
scheduler_system.py:12:4: W0603: Using the global statement (global-statement)
scheduler_system.py:13:4: C0206: Consider iterating with .items() (consider-using-dict-items)
scheduler_system.py:15:29: C0209: Formatting a regular string which could be an f-string (consider-using-f-string)
scheduler_system.py:22:0: C0116: Missing function or method docstring (missing-function-docstring)
scheduler_system.py:27:4: W0702: No exception type(s) specified (bare-except)
scheduler_system.py:30:0: C0116: Missing function or method docstring (missing-function-docstring)
scheduler_system.py:34:0: C0116: Missing function or method docstring (missing-function-docstring)
scheduler_system.py:35:8: W1514: Using open without explicitly specifying an encoding (unspecified-encoding)
scheduler_system.py:35:8: R1732: Consider using 'with' for resource-allocating operations (consider-using-with)
scheduler_system.py:39:0: C0116: Missing function or method docstring (missing-function-docstring)
scheduler_system.py:39:0: C0103: Function name "logConflicts" doesn't conform to snake_case naming style (invalid-name)
scheduler_system.py:40:8: W1514: Using open without explicitly specifying an encoding (unspecified-encoding)
scheduler_system.py:40:8: R1732: Consider using 'with' for resource-allocating operations (consider-using-with)
scheduler_system.py:45:0: C0116: Missing function or method docstring (missing-function-docstring)
scheduler_system.py:46:4: W0612: Unused variable 'a' (unused-variable)
scheduler_system.py:47:4: W0612: Unused variable 'a_more' (unused-variable)
scheduler_system.py:49:4: W0612: Unused variable 'c' (unused-variable)

-----------------------------------
Your code has been rated at 5.60/10
```

Figure 3. Bandit before corrections

```
(venv) PS C:\Users\Owner\L5_12_room_scheduler> bandit -r scheduler_system.py > bandit_before.txt
[main]  INFO    profile include tests: None
[main]  INFO    profile exclude tests: None
[main]  INFO    cli include tests: None
[main]  INFO    cli exclude tests: None
[main]  INFO    running on Python 3.11.8
```

Figure 4. Flake8 before corrections

```
(venv) PS C:\Users\Owner\L5_12_room_scheduler> flake8 scheduler_system.py > flake8_before.txt
(venv) PS C:\Users\Owner\L5_12_room_scheduler> Get-Content .\flake8_before.txt
scheduler_system.py:11:1: E302 expected 2 blank lines, found 1
scheduler_system.py:14:80: E501 line too long (87 > 79 characters)
scheduler_system.py:18:80: E501 line too long (102 > 79 characters)
scheduler_system.py:22:1: E302 expected 2 blank lines, found 1
scheduler_system.py:27:5: E722 do not use bare 'except'
scheduler_system.py:30:1: E302 expected 2 blank lines, found 1
scheduler_system.py:34:1: E302 expected 2 blank lines, found 1
scheduler_system.py:39:1: E302 expected 2 blank lines, found 1
scheduler_system.py:45:1: E302 expected 2 blank lines, found 1
scheduler_system.py:46:5: F841 local variable 'a' is assigned to but never used
scheduler_system.py:47:5: F841 local variable 'a_more' is assigned to but never used
scheduler_system.py:49:5: F841 local variable 'c' is assigned to but never used
scheduler_system.py:59:1: E305 expected 2 blank lines after class or function definition, found 1
```

6. Issues Identified and Corrections

The following changes were made to address the reported problems and improve the program's reliability, maintainability and security.

Issue	Correction applied

Unsafe shell command execution	Removed the os.system-based shell command used to display the log file. The log is now read and printed using Python file handling.
Bare exception handling	Replaced broad exception handling with specific exception types and raised a descriptive ValueError with exception chaining.
Mutable default argument	Changed the equipment_list default to None and initialized a new list inside the function.
Time-slot validation	Added type checking and validation to ensure that time slots follow the expected HH-HH format and that the end hour is later than the start hour.
File handling	Used context managers and explicit UTF-8 encoding when reading and writing files.
Naming and documentation	Improved function naming and added module/function docstrings.
Formatting and unused variables	Corrected whitespace and line-length issues and removed unused variables.

7. Final Static Analysis Results

After the corrections, all three tools were executed again. The final results show that the updated source achieved a Pylint score of 10.00/10, Bandit reported no remaining issues, and Flake8 produced no violations.

Tool	Final result	Status
Pylint	10.00/10	Passed
Bandit	No issues identified (0)	Passed
Flake8	No output / 0 violations	Passed

7.1 Final Tool Evidence

Figure 5. Pylint after corrections

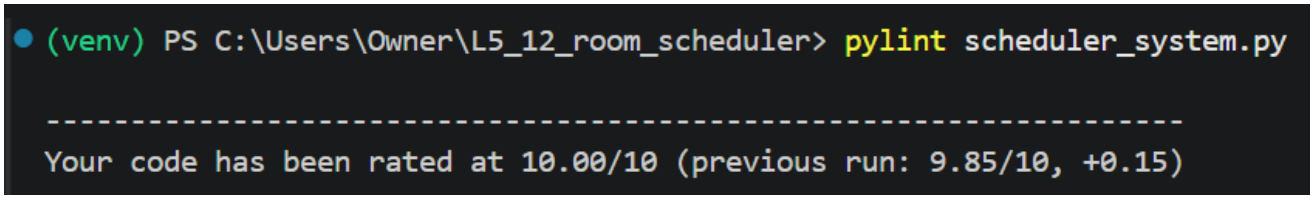


Figure 6. Bandit after corrections

```
(venv) PS C:\Users\Owner\L5_12_room_scheduler> bandit -r scheduler_system.py
[main]  INFO    profile include tests: None
[main]  INFO    profile exclude tests: None
[main]  INFO    cli include tests: None
[main]  INFO    cli exclude tests: None
[main]  INFO    running on Python 3.11.8
Run started:2026-10-31 03:10:34.297891+00:00

Test results:
        No issues identified.

Code scanned:
        Total lines of code: 93
        Total lines skipped (#nosec): 0

Run metrics:
        Total issues (by severity):
                Undefined: 0
                Low: 0
                Medium: 0
                High: 0
        Total issues (by confidence):
                Undefined: 0
                Low: 0
                Medium: 0
                High: 0
Files skipped (0):
```

Figure 7. Flake8 after corrections

```
(venv) PS C:\Users\Owner\L5_12_room_scheduler> flake8 scheduler_system.py
(venv) PS C:\Users\Owner\L5_12_room_scheduler>
```

8. Program Execution and Functional Verification

The updated scheduler_system.py was executed after the changes. The program completed successfully, calculated the duration of a valid time slot, recorded a duplicate-booking conflict, allocated equipment, exported the reservation schedule and displayed the remaining equipment and reservations. This check was used to confirm that the static analysis corrections did not prevent the demonstration program from running.

Figure 8. Final program execution

```
(venv) PS C:\Users\Owner\L5_12_room_scheduler> python scheduler_system.py
Slot length for reservation: 2
Slot for Board-2, like 10-12 (Enter for 10-12):
2026-10-31 08:26:29.685985 Board-1 double-booked at 09-11
2026-10-31 08:42:13.594065 Board-1 double-booked at 09-11

Equipment left: {'projector': 1, 'microphone': 4, 'webcam': 0}
Reservations: {1: {'room': 'Board-1', 'slot': '09-11', 'equipment': []}, 2: {'room': 'Board-1', 'slot': '09-11', 'equipment': []}, 3: {'room': 'Huddle-3', 'slot':
'12-14', 'equipment': []}, 4: {'room': 'Huddle-3', 'slot': '15-16', 'equipment': []}, 5: {'room': 'Board-2', 'slot': '10-12', 'equipment': ['projector', 'webcam']}
}
```

9. Submission Structure and Version Control

The Lab 5 submission is stored in the team repository under Lab5/PS4_PES1UG24AM134_Khushi/. The folder contains the cleaned source code, issue summary, reflection and before/after reports for all three analysis tools.

- scheduler_system.py
- issues.md
- reflection.md
- pylint_before.txt and pylint_after.txt
- bandit_before.txt and bandit_after.txt
- flake8_before.txt and flake8_after.txt

The work was committed in three Lab 5 commits and pushed to the team's main branch. The commits organize the Pylint-related source and issue documentation, the Bandit analysis reports, and the Flake8 verification and reflection. The final source file contains the combined corrections, including the Bandit and Flake8 fixes.

10. Reflection

What is static code analysis and why is it important?

Static code analysis examines source code without requiring the program to be executed. It helps identify defects, unsafe coding practices and style problems early, making the program easier to maintain and more reliable.

What are the differences between Pylint, Bandit and Flake8?

Pylint evaluates broader code quality and reports issues such as missing documentation, unused variables and naming problems. Bandit focuses on common security risks in Python code. Flake8 checks style and formatting conventions and reports violations such as incorrect spacing, excessive line length and unused imports or variables.

What issues were identified and fixed in this program?

The work addressed unsafe shell execution, broad exception handling, a mutable default argument, inadequate time-slot validation, file-handling practices, naming/documentation issues and formatting violations.

How did static code analysis improve the program?

The tools provided specific, actionable feedback. Applying the corrections improved readability, validation, error handling and security. The final checks reported a Pylint score of 10.00/10, zero Bandit findings and zero Flake8 violations.

11. Conclusion

The Lab 5 static code analysis exercise was completed by analysing the Room Scheduler program with Pylint, Bandit and Flake8, applying corrections and verifying the final results. The updated program executed successfully, achieved a Pylint score of 10.00/10, had no remaining Bandit findings and produced no Flake8 violations. The source code, issue summary, reflection and tool reports have been committed and pushed to the team repository.